

Applied Program Evaluation Using Stata

A Practical Workflow from Data to Deliverables

Authors: Eric Booth & Elizabeth Teas

Description of project

What is this book about?

Applied Program Evaluation Using Stata teaches the whole job an embedded analyst does for a nonprofit, foundation, or government agency, from an empty project folder to a delivered result. Most statistics texts start after the data is clean. This one starts before it arrives and keeps going after the model runs.

Readers learn to run the entire project lifecycle from Stata: pull raw data from public APIs and from agencies that publish nothing but a download button, combine program and survey data with administrative records, link longitudinal records that share no identifier, make a claim at the level of rigor the data supports, and deliver the result as a formatted workbook, a live Google Sheet, or a self-contained HTML portal that opens behind a locked-down firewall. A chapter on large language models treats them as a coding partner working under guardrails rather than a source of numbers, and we wrote it so the guidance survives the next model release rather than describing today's tools.

Why should it exist? Why is it new?

We cover the first mile and the last mile of an evaluation, the parts most Stata resources skip. We use every one of these skills in our own policy and program work, and we teach parts of it in workshops for policy PhD students and early-career researchers. What our audiences ask for has changed: funders now expect an interactive dashboard rather than a static PDF, the difference-in-differences literature has overturned what a two-way fixed-effects regression can claim, and everyone feels pressure to use AI without a clear account of the privacy, accuracy, and reproducibility risks. We put those demands into one reproducible Stata pipeline.

Where it fits in the Stata publication universe: excellent texts already cover microeconomics (Cameron and Trivedi) and workflow discipline (Long). What no Stata book covers is *delivery* and *operations*, the capabilities Stata users routinely say they want and reach for R or Python to get. A *Stata Journal* article could cover any one of the commands we introduce, and one on the Google tools is in progress. The value here is the connection between them: an API pull feeds a panel, the panel feeds an empirical-Bayes shrinkage estimator, and the estimator feeds a scheduled HTML dashboard that needs no server. Showing that chain end to end takes a book.

The book's toolkit appendix lists every package it uses. Several of our own commands are already on SSC as of this month, and we expect the rest to be posted there before the book comes out.

Why are you the one to write it?

We have spent our careers in applied evaluation and policy, currently at the Texas 2036 policy think tank and at Far Harbor, LLC, a private research and evaluation consulting firm. We also build the tools. The Stata packages this book teaches are ones we wrote because our own projects needed them, including `webapi`, `googlesheets`, `statashiny`, `webdoc2`, `rateshrink`, and `convertanything`. Between us we have spent years working inside restrictive IT environments, making defensible claims from samples smaller than we would like, and building data systems that practitioners keep using after the contract ends.

Why is now the time to publish it?

Three shifts make the book timely:

- 1. Evaluators are already using AI, often unsafely.** Analysts paste administrative data into a chat window, ask a model to do statistical work it is poorly matched for, and cannot reproduce what comes back. Chapter 13 gives a Stata-centric framework for using a model safely: the LLM drafts and annotates code one checkable step at a time, Stata computes and logs every result, and `assert` tripwires decide whether a step passed. We know of no other Stata treatment of this.
- 2. The difference-in-differences literature has moved.** Applied researchers are working to catch up with Callaway and Sant'Anna and with synthetic controls, and they need to use these methods on Monday. We give a practical translation using `csdid`, with the diagnostics that tell a reader when the design is appropriate and when it is not.
- 3. Stakeholders expect interactive deliverables.** We show how to write them from do-files: Socrata and other API pipelines, interactive charts, and JavaScript-powered dashboards, all through Stata wrappers so the reader writes no HTML or JavaScript. `webapi` runs a small helper against the `python3` already on the machine, using only the standard library, so there is nothing to `pip install` and no Stata-Python configuration to set up. The whole path from download to a published dashboard runs from do-files, with no license for an external BI tool.

Who makes up the core audience and why will they find it appealing?

The Core Audience:

- 1. Practitioner-researchers and evaluators:** analysts in nonprofits, think tanks, foundations, school districts, and government agencies doing Monitoring, Evaluation, and Learning (MEL). We teach free and grant-sponsored workshops for these audiences, and they ask us for these packages, features, and code examples.
- 2. Applied data scientists:** researchers modernizing a Stata workflow with APIs, scraping, and dashboards that need no server.
- 3. Implementation scientists:** academics and practitioners doing real-time program monitoring and fidelity work, often embedded with the program while the study runs.

Their situations, and how this book helps:

These readers face chaotic data drops, strict data-use agreements, restrictive firewalls, and audiences who will not read a regression table. On a Thursday afternoon someone asks whether the program is reaching the students it promised to reach, and the answer is due Monday.

We give them runnable code to:

- Ingest dozens of mismatched Excel files in one call (`convertanything`).
- Stabilize noisy small-denominator rates so a tiny clinic is not punished by an accident of sample size (`rateshrink`).
- Deliver in the format the client already uses, a formatted Excel workbook or a live Google Sheet, without typing a number by hand (`googlesheets`).
- Build interactive HTML that opens inside an agency firewall, including charts that draw with the internet off (`sparkta2` , `statashiny` , `webdoc2` , `googlechart`).

Assumed knowledge:

We assume basic-to-intermediate Stata: the reader knows what a do-file is, can merge datasets, and can run a regression. The book builds naturally on Scott Long's workflow book, and our `projectbuilder` package follows many of the same principles while automating the documentation, error-checking, and path control that his book has the reader hand-code.

We do *not* assume a background in computer science, web development, or advanced econometrics. We teach the vocabulary and the few skills a reader needs to know why and when to touch an API, a JSON or Parquet file, an HTML page, or a causal design, but we do not ask anyone to become an expert in those mechanics. The wrappers exist so that an evaluator can ship a Stata dashboard with an interactive map without writing HTML or JavaScript.

Annotated Table of Contents

The book has five parts, following a project's natural lifecycle from an empty folder to a delivered portal.

PART I: THE EMBEDDED EVALUATOR WORKFLOW

1. The embedded evaluator

Point: Defines the embedded evaluator's role and the book's four principles: replicable, extensible, accessible, and actionable.

Contribution: Sets the professional and ethical foundation, and gives readers the vocabulary (Plan-Do-Study-Act, utilization-focused evaluation) to defend their independence and negotiate a data charter with a client.

2. Setting up a project that survives deadlines

Point: Building a folder structure and a control file so a project can be handed off or rerun years later.

Contribution: Introduces the numbered pipeline, package pinning, the tracking spine, and benchmarked speed habits, so a reader's code still runs on a colleague's laptop. Scaffolds the whole layout in one command with `projectbuilder`.

PART II: GETTING THE DATA

3. Downloading public data through APIs

Point: The anatomy of an API request, and how to pull the denominators (Census, BLS, CDC) an evaluation needs without manual downloads.

Contribution: Covers `getcensus`, `educationdata`, `import fred`, and our `webapi`, which flattens nested JSON into a Stata dataset by calling a standard-library Python helper, with nothing for the reader to install or configure.

4. Harvesting data when there is no API

Point: Automating ingestion when an agency offers only a download button or a messy shared-drive drop.

Contribution: Covers responsible scraping, streaming files too large to load, and `convertanything` for a folder tree of mixed formats.

5. Working with survey platforms

Point: Connecting Stata to live survey platforms (Qualtrics, REDCap) to watch response rates while the survey is open.

Contribution: Teaches total survey error, scale construction (`likertscale`), nonresponse bias diagnostics, and cross-wave codebooks.

6. Building longitudinal data you can trust

Point: Linking administrative records with no shared identifier and reshaping them into an analysis-ready panel.

Contribution: Covers probabilistic record linkage (cleaning, blocking, Jaro-Winkler scor-

ing), schema drift, declaring a panel with `xtset`, and reporting missing data honestly.

PART III: TURNING DATA INTO EVIDENCE

7. Making numbers trustworthy

Point: The safeguards that come before a claim: reliability, survey weighting, and stabilizing noisy rates.

Contribution: Introduces `rateshrink` for empirical-Bayes shrinkage, `conformalpred` for distribution-free prediction intervals, and power analysis framed as the minimum detectable effect a funder should hear about before data collection starts.

8. From differences to defensible claims

Point: Matching the claim to the design, centered on modern difference-in-differences.

Contribution: Translates the new DiD econometrics (`csdid`) for practitioners: how two-way fixed effects goes wrong, how to read an event study, how to choose a comparison group you can defend, and how to simulate return on investment (`roisim`).

PART IV: COMMUNICATING RESULTS

9. Graphing for busy readers

Point: Static graphics built for how people actually read a chart.

Contribution: Applies the Cleveland-McGill perceptual rankings to Stata graphics, and covers small multiples, coefficient plots (`coefplot`), one-command category comparisons with intervals (`statplot`), and captions that state the finding.

10. Building interactive charts and infographics

Point: Interactive web charts, maps, and animations written from Stata rather than from a BI tool.

Contribution: Covers `googlechart` for online interactive pages and `sparkta2` for the case a public agency actually presents, a chart that draws with the internet off and maps below the state level.

11. Delivering through spreadsheets

Point: Delivering in the client's own format without giving up reproducibility.

Contribution: Teaches the never-typed-number rule, using `putexcel`, `collect`, and `googlesheets` to write estimates straight into formatted workbooks and live shared Sheets.

12. Publishing self-contained reports and portals

Point: Wrapping narrative, code, and interactive charts into one portal a client can open offline.

Contribution: Uses `webdoc2` and `statashiny` to compile Stata output into a dashboard that needs no server, and stamps every release with a version, a data vintage, and a Git commit hash so an auditor can rebuild the exact numbers.

PART V: SUSTAINING THE PRACTICE

13. Using AI without getting burned

Point: A workable framework for using a large language model in an evaluation without losing reproducibility or leaking data.

Contribution: Keeps Stata as the system of record and the LLM as a coding partner. Introduces `llmsieve` for multi-model agreement, a privacy gate at intake, and defenses against prompt injection when classifying free-text case notes.

14. Sharing data and results safely

Point: Preparing files for release without re-identifying the people in them.

Contribution: Teaches k -anonymity scanning (`riskscan`), primary and complementary cell suppression (`suppress`), and a disclosure-review checklist a lawyer and an analyst can sign together.

15. Turning scripts into shared tools

Point: Turning a repeated do-file block into a documented, installable Stata package.

Contribution: Shows how to write the `.ado` and its help file, distribute through GitHub with `net install`, write a test battery before the command it tests, drop into Mata or Python only where it pays, and leave behind a replication archive.

Sample Chapter

We drafted the manuscript in $\text{\LaTeX}$ using the `kaobook` class, which gives a wide margin for notes and figures. The compiled PDF runs to 328 pages across fifteen chapters and four appendices, with live code, real statistical output, and the figures the companion do-files generate. We can send the whole draft, or any single chapter, as PDF or $\text{\TeX}$.

About the Authors

Eric A. Booth works in applied evaluation and data systems in the public policy and nonprofit sectors, currently at Texas 2036. He has a long record of teaching and of writing open-source Stata packages that connect statistical analysis to modern data engineering, with an emphasis on API access, reproducibility, and workflow automation.

Elizabeth Teas is an applied researcher and evaluator at Far Harbor, LLC. She specializes in implementation science and program evaluation, and in turning statistical findings into strategies community organizations, foundations, and government agencies can act on.

A note on rights and affiliation. We name our current positions above as professional background only. This book is our own work, drawn on a career's worth of accumulated practice rather than on any employer's project, and it is not written under, sponsored by, or endorsed by either organization. Neither employer holds any rights in the manuscript, the companion code, or the software packages, and neither would be a party to a publishing agreement. We contract as individuals.

Timetable

We have drafted all fifteen chapters and four appendices, they compile clean, and every number printed in the book comes from a companion do-file we can rerun on request. The current draft is 328 pages in the `kaobook` layout; converting to the Stata Press layout would tighten it, and we expect some reference material would move to a technical appendix or a companion website. We have written and tested every custom package the book teaches, and all but `sparkta2` are already installable.

- **Final manuscript delivery:** within three months of contract, plus time for peer review, layout conversion, and copyediting. Because parts of this material track fast-moving tools, we would like the option to revise close to press time.
- **Target publication:** 2027.